Symbiosis Institute of Technology, Nagpur

INDUSTRY INTERNSHIP REPORT

Submitted in partial fulfillment of requirement for the award of degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Kritika Nimje
PRN: 22070521193

Industry Guide

Mrs. Ruchita Charmode
Assistant Manager

at

InfoCepts Technologies Pvt. Ltd.

Institute Guide

Dr. Smita Nirkhi Singh
Assistant Professor

May 2026

॥वसुधैव कुटुम्बकम्॥

Symbiosis International (Deemed University), Pune

॥वसुधैव कुटुम्बकम्॥
SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)
(Established under section 3 of the UGC Act, 1956)
Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

DECLARATION

I, hereby declare that the Industry Internship report submitted herein has been carried out by me in **(InfoCepts Technologies Pvt. Ltd.)** towards partial fulfillment of requirement for the award of Degree of Bachelor of Technology in Computer Science and Engineering. The work is original and has not been submitted earlier as a whole or in part for the award of any degree / diploma at this or any other Institution / University.

I hereby assign all copyright rights in this work, including any revised or expanded derivative works, to Symbiosis Institute of Technology, Nagpur. Any content sourced from references, manuals, or other external materials is duly acknowledged and disclaimed.

Name of Student: **Kritika Nimje**
PRN: **22070521193**
Signature:

Place: Nagpur
Date: April 20, 2026

॥वसुधैव कुटुम्बकम्॥
SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)
(Established under section 3 of the UGC Act, 1956)
Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

CERTIFICATE

This is to certify that the Industry Internship Report, titled **"Case Study(s) / Project Title,"** has been carried out under our supervision at **[InfoCepts Technologies Pvt. Ltd.]** by **[Kritika Nimje]** in partial fulfillment of the requirements for the Bachelor of Technology in Computer Science and Engineering. The work submitted is comprehensive, complete and fit for evaluation.

Mrs. Ruchita Charmode
Industry Guide

Dr. Smita Nirkhi Singh
Institute Guide

Dr. Priya Dasarwar
Internship Incharge

Dr. Sagarkumar S. Badhiye
Head of Department

Dr. Nitin Rakesh
Professor and Director

(Certificate from industry / organization on industry / organization letter head describing duration and project work done, project implemented, project sponsored by industry, achievement & stipend details)

(On Industry / Organization Letter Head)

Certificate (Sample)

This is to certify that Mr. /Ms. XYZ student of Computer Science and Engineering from Symbiosis Institute of Technology, Nagpur has completed Internship successfully from DD/MM/YYYY to DD/MM/YYYY. During this period, he/she has shown good interest in the assignment/works given to him/her and worked hard. During his/her tenure of internship, he/she was hard working and focused on activities assigned to them.

Students have worked during internship period on following project under the guidance of Mr. ABC, (Designation & Department).

1. A
2. B
3. C

Regards,

Signature Date:
Name of HR Head
Designation
Industry / Organization Name & Address

NOTE: Reference no/ Document Number is MUST in this certificate

(On Industry / Organization Letter Head)

Certificate

Ref.No./EBOS/Project/2024-25/007 (Example) **Date:**

Cost of Industrial Solution Certificate (Mandatory)

This is to certify that Mr. /Ms. ABC students of Computer Science and Engineering from Symbiosis Institute of Technology, Nagpur has completed Internship successfully from DD/MM/YYYY to DD/MM/YYYY. During this period, they have shown good interest in the assignment/works given to them and worked hard.

Students have worked during internship period on following project/industrial problem under the guidance of Mr. ABC,(Designation & Department).

TITLE OF THE PROJECT WORK/DOMAIN/INDUSTRIAL PROBLEM / CASE STUDIES Industry has spent Rs……*** amount on their ideas/industries problem, which they have successfully implemented.

(*** ENTER NIL AMOUNT IF INDUSTRY DOES NOT AGREE TO MENTION AMOUNT)

Regards,

Signature
Name of HR Head
Designation
Industry / Organization Name & Address, SEAL MUST

NOTE: Reference no/ Document Number is MUST in this certificate

(On Industry / Organization Letter Head)

Certificate

Ref.No./EBOS/Project/2024-25/007 (Example) **Date:**

Savings to Industry Certificate (Mandatory)

This is to certify that Mr./Ms. ABC students of Computer Science and Engineering from Symbiosis Institute of Technology, Nagpur has completed Internship successfully from DD/MM/YYYY to DD/MM/YYYY. During this period, they have shown good interest in the assignment/works given to them and worked hard.

Students have worked during internship period on following project/industrial problem under the guidance of Mr. ABC, (Designation & Department).

TITLE OF THE PROJECT WORK/DOMAIN/INDUSTRIAL PROBLEM/ CASE STUDIES Project work submitted by them has the potential to save cost up to Rs …. Lakh/year. Also, they were entitled for stipend of Rs. ……. *** /- per month along with canteen, transportation & accommodation facilities.

(*** ENTER NIL IN AMOUNT, IF INDUSTRY DOES NOT AGREE TO MENTION AMOUNT)

Regards,

Signature
Name of HR Head
Designation
Industry / Organization Name & Address, SEAL MUST

॥वसुधैव कुटुम्बकम्॥
SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)
(Established under section 3 of the UGC Act, 1956)
Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

ACKNOWLEDGEMENT

I express my sincere gratitude to **[InfoCepts Technologies Pvt. Ltd.]** for providing me with the opportunity to complete my internship in **[Data Engineering]** and gain valuable industry experience. This internship has been a highly enriching journey that helped me develop both technical and professional skills.

I am deeply grateful to my mentor/supervisor, **[Mrs. Ruchita Charmode]**, for their constant guidance, insightful feedback, and encouragement throughout the internship. Their expertise and support played a crucial role in enhancing my learning experience. I also extend my appreciation to my colleagues and team members at **[InfoCepts Technologies Pvt. Ltd.]** for their cooperation, assistance, and for making my internship a pleasant and productive experience.

I am immensely grateful to my academic institution, Symbiosis Institute of Technology, Nagpur, for facilitating this internship opportunity. I extend my heartfelt appreciation to our Director, **Dr. Nitin Rakesh**, for his continuous support and encouragement in providing students with valuable industry exposure. I would also like to express my deepest gratitude to Dr. Sagarkumar S. Badhiye, Head of the Department, Computer Science and Engineering, for his constant support, encouragement, and invaluable guidance throughout my academic journey.

A special note of thanks goes to the **Training and Placement Department** for their efforts in providing internship opportunities and career guidance. I would also like to express my sincere gratitude to **Dr. Priya Dasarwar**, Internship In-Charge, for her support in coordinating the internship process and ensuring a smooth learning experience.

Furthermore, I sincerely appreciate the guidance and motivation provided by my Institute Mentor, **[Institute Mentor's Name]**, whose valuable insights and feedback have greatly contributed to my professional growth during this internship.

Lastly, I would like to acknowledge my family and friends for their unwavering support and encouragement during this period.

Thank you all for your invaluable contributions to my learning and professional growth.

ABSTRACT

The report summarizes the work performed during an industry internship within the data engineering and analytics domain, specifically the design of scalable and efficient data processing systems. The internship involved the implementation of three large-scale projects in food-tech, retail, and pharmaceutical sectors. All projects covered practical problems of data quality, integration, and analytics.

The initial project was the implementation of an ETL pipeline on Databricks to process semi-structured data with the help of the Medallion Architecture. Cleansing, deduplication, and incremental loading mechanisms ensured data quality, and Delta Lake and Unity Catalog were utilized to ensure reliability, governance, and data lineage.

The second project, the *RetailPulse Analytics Platform*, was aimed at building an AWS-based end-to-end data pipeline. The solution combined data across multiple sources with the help of Apache Spark on EMR, processed datasets were stored in an Amazon S3 data lake, and analytics were enabled with the help of AWS Glue and Athena. Amazon QuickSight was used to create interactive dashboards, along with a serverless real-time anomaly detection system.

The third project dealt with pharmaceutical supply chain issues using real-time streaming with Amazon Kinesis and Spark Structured Streaming. Both continuous monitoring and batch-level traceability were supported by the architecture, which was based on a unified streaming and batch processing design, allowing quick discovery of operational and compliance problems.

Altogether, the internship gave me practical experience in the field of modern data engineering techniques and architectural design patterns, with a focus on scalable platforms, data quality, and real-time analytics to address complex business issues.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION TO COMPANY

1.1 About the Company

Infocepts is a global data and analytics consulting organization that specializes in building scalable data platforms, cloud solutions, and advanced analytics systems for enterprises.

1.2 Location

Infocepts is an organization having a global distributed structure with its corporate base in **Naperville, Illinois, USA**. It has its main base of delivery and engineering in **Nagpur, Maharashtra, India**, which serves as the center of data engineering, cloud platform development, and analytics delivery to international clients. There are also other offices in **Pune, Maharashtra** and **Hyderabad, Telangana** that support consulting, pre-sales, and specialized AI/ML practices. The company also has a presence in **New Jersey, USA** to engage with East Coast clients.

The Nagpur delivery center, where this internship took place, has cross-functional teams structured around data engineering, cloud architecture, BI and visualization, quality assurance, and program management. The center operates across time zones and collaborates in real-time with counterparts in the United States and Europe on enterprise-grade data platform projects.

1.3 Operational Structure

Infocepts is structured into competency-based practice areas that work closely to provide integrated data solutions. The major functional departments are:

Each practice area is led by a Principal Architect or Practice Head and supported by senior engineers, data engineers, analysts, and associate engineers. Agile methodologies are followed, typically with two-week sprints, including daily stand-ups, sprint reviews, and retrospectives using tools such as Jira, Confluence, and Microsoft Teams.

Interns are embedded directly into engineering squads and contribute to active client deliverables under the guidance of senior engineers and industry mentors.

Table 1.1: Infocepts Practice Areas and Functions

Sr. No.	Practice Area	Core Function
1	Data Engineering	ETL/ELT pipelines, lakehouse architecture, batch and streaming processing
2	Cloud	AWS, Azure, and GCP infrastructure design and migration
3	Analytics & BI	Dashboards, self-service analytics, KPI frameworks
4	AI & ML	Predictive modeling, NLP, and MLOps pipelines
5	Data Governance	Data quality, lineage, compliance, and cataloging
6	Consulting	Strategy, roadmap definition, and architecture review

1.4 Vision and Mission

The vision of Infocepts is to become a trusted global partner for enterprises in their data-driven transformation journey, enabling organizations to extract strategic value from their data assets.

The company focuses on delivering scalable, intelligent, and cloud-native data solutions that empower organizations to make faster and more informed decisions. It combines strong technical expertise with a consulting-driven approach, ensuring that every data platform delivers measurable business outcomes and long-term sustainability.

The organizational culture emphasizes continuous learning, customer-centricity, engineering excellence, and collaborative innovation. Internal initiatives such as the Data Engineering Guild, technical workshops, certification programs (AWS, Databricks, Snowflake), and mentorship programs support the professional growth of employees.

1.5 Products and Software Solutions

Infocepts is not a traditional product-based company; instead, it offers proprietary accelerators, frameworks, and reusable solution templates that enhance its service offerings. Key intellectual property assets include:

In addition to these accelerators, Infocepts delivers customized solutions using industry-leading platforms such as **Databricks Lakehouse**, **AWS Glue**, **Amazon Redshift**, **Snowflake**, **Apache Spark Structured Streaming**, **Amazon Kinesis Data Streams**, and **dbt (data build tool)**. The company also partners with tools like Tableau, MicroStrategy, and Alteryx for analytics and visualization.

The intern was actively engaged in three client-facing data engineering projects dur-

Table 1.2: Infocepts Proprietary Accelerators and Tools

Sr. No.	Accelerator / Product	Description
1	DataAscent	End-to-end cloud data migration system supporting lift-and-shift and re-architecture patterns across AWS, Azure, and GCP
2	PipelineIQ	Modular ETL/ELT template library built on Databricks and Apache Spark for rapid pipeline development
3	InsightHub	Self-service analytics and BI enablement layer integrating with Tableau, Power BI, and Looker
4	DataQualityX	Data profiling and quality validation platform with rule-based validation and automated alerts
5	StreamEdge	Real-time streaming pipeline accelerator based on Apache Kafka and Amazon Kinesis

ing the internship period, spanning food-tech, retail, and pharmaceutical sectors. Each project leveraged different combinations of these technologies and accelerators. The technical implementations, system architectures, and outcomes are discussed in detail in the subsequent chapters.

2. PROJECT WORK / CASE STUDY

2.1 Project 1: Zomato ETL Pipeline on Databricks

1.1 Introduction

The food-tech sector produces huge amounts of semi-structured data on a daily basis, including restaurant descriptions, customer reviews, food categories, location capabilities, and delivery attributes. In platforms such as Zomato, this data forms the foundation of business intelligence, enabling country-level benchmarking, cuisine trend analysis, pricing strategy, and operational decision-making.

However, raw data in JSON format cannot be directly used for analytical purposes without a stable, governed, and scalable transformation pipeline.

The objective of this project, carried out during the internship at Infocepts, was the design and development of a production-grade ETL (Extract, Transform, Load) pipeline on Databricks Free Edition using modern data engineering best practices. The Zomato dataset, being semi-structured, is ingested into the pipeline, processed across multiple layers, and stored as Delta Lake tables under Unity Catalog. The final outputs are curated, partitioned datasets ready for downstream analytics and reporting.

The solution is based on the **Medallion Architecture (Bronze $\rightarrow$ Silver $\rightarrow$ Gold)**, a widely adopted lakehouse design pattern that enforces separation of concerns and progressive data quality improvement at each stage. The pipeline is orchestrated through a centralized driver notebook that manages execution flow, maintains audit logs, enforces concurrency control through locking mechanisms, and supports multiple load strategies such as manual, historical, and scheduled execution.

1.2 Problem Identification

Zomato restaurant data is received in the form of nested and multi-level JSON files containing lists of restaurant objects. Each object includes attributes such as location, cuisine, pricing, ratings, and delivery information. In its raw state, this data presents several engineering and governance challenges that prevent direct analytical consumption:

1. **Semi-structured Format:** JSON files are deeply nested, with restaurant attributes embedded within multi-level hierarchies (e.g., `restaurant.location.city`,

`restaurant.user_rating.aggregate_rating`). Without flattening and schema enforcement, consistent data access is not possible.

2. **Lack of Data Quality Controls:** Raw data often contains NULL values, missing cuisines, inconsistent string formats, and improperly encoded binary flags. Direct usage in reporting layers leads to inaccurate aggregations and unreliable insights.

3. **Duplicate Records:** Reprocessing or repeated ingestion may introduce duplicate records for the same restaurant and file date, leading to inflated metrics and incorrect business KPIs.

4. **Weak Governance and Lineage:** Without centralized metadata management, tables become difficult to discover, audit, and control. There is no structured mechanism to track data origin, transformation history, or access patterns.

5. **No Incremental Load Support:** A full refresh strategy for every pipeline execution is inefficient and computationally expensive. Enterprise-grade systems require idempotent and incremental data loading mechanisms.

6. **Lack of Operational Safety:** Concurrent pipeline executions without proper locking mechanisms may lead to conflicting writes and data corruption. Additionally, the absence of audit logs makes debugging and monitoring difficult.

These challenges define the core engineering problem addressed by this project.

1.3 Objectives

The main goals that were set in this project were as follows:

- Run a multi-stage, modular ETL pipeline using Databricks based on the Medallion Architecture (Bronze $\rightarrow$ Silver $\rightarrow$ Gold).
- Use Unity Catalog for centralized governance, schema management, and table discovery across all pipeline assets.
- Use Delta Lake as the storage layer to ensure ACID-compliant writes, scalable storage, and time-travel capabilities.
- Implement comprehensive data quality rules such as NULL filtering, deduplication, string standardization, and schema enforcement at the transformation stage.
- Compute business-critical columns such as `m_rating_colour` (rating band classification) and `m_cuisines` (cuisine category) to enrich analytical datasets.

- Support incremental, manual, and historical load strategies to enable operational flexibility, backfills, and recovery scenarios.
- Introduce audit logging through a dedicated log table (`zomato_summary_log`) and implement a locking mechanism to ensure operational observability and concurrency safety.
- Design the pipeline for daily execution, generating country-wise and date-wise partitioned summary tables for BI consumption.

1.4 Architecture Overview

The pipeline is built on the Databricks Lakehouse platform and follows the Medallion Architecture, organized into three Delta Lake layers. The entire workflow is governed using Unity Catalog and orchestrated through a centralized driver notebook. Figure 2.1 illustrates the high-level architecture.

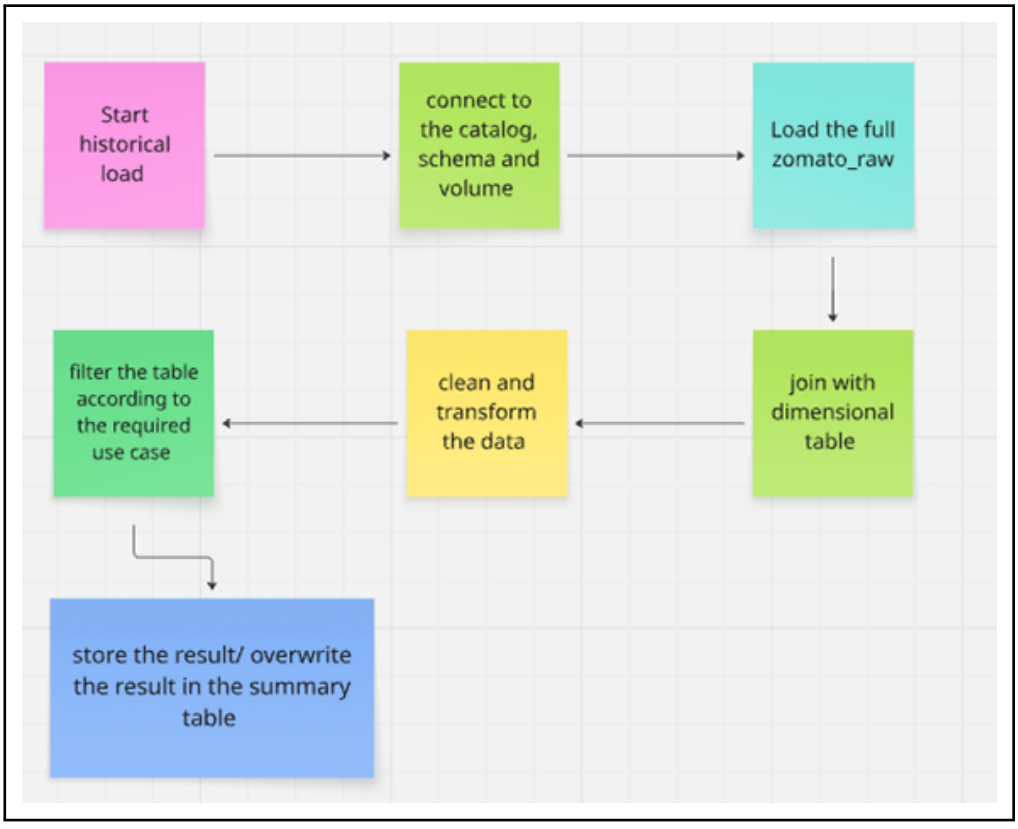


Figure 2.1: Zomato ETL Pipeline Medallion Architecture on Databricks

The architecture is organized into four logical layers:

Tier 1 – Ingestion Layer (Databricks Volumes): Raw JSON files are ingested into the `zomato_raw_files` folder within a Unity Catalog-managed volume (`zomato_files`).

The data is stored in a structured format including staged JSON files, converted CSV files, archives, and execution logs.

Tier 2 – Bronze Layer (`zomato_raw`): Structured CSV data (converted from JSON in the ingestion module) is loaded into the Delta table `zomato.etl.zomato_raw`. This layer stores minimally transformed data, partitioned by file date, and acts as the immutable source of truth for downstream processing.

Tier 3 – Dimension Layer (`dim_country`): A country dimension table is created using a reference CSV file. This table enriches raw records with human-readable country names and enables partitioning by country in downstream layers.

Tier 4 – Gold Layer (`zomato_summary`): The final analytics-ready table is generated by applying business rules such as filtering, deduplication, null handling, computed column derivation, and audit column addition. The output is stored as a Delta table partitioned by `p_filedate` and `p_country_name`.

All schemas, tables, and access controls are managed using Unity Catalog under the `zomato` catalog and `etl` schema, ensuring proper governance across the pipeline.

Table 2.1: Medallion Architecture Layer Mapping – Zomato ETL Pipeline

Layer	Table / Asset	Format	Purpose
Bronze	`zomato.etl.zomato_raw`	Delta Lake	Raw ingestion, partitioned by file date
Dimension	`zomato.etl.dim_country`	Delta Lake	Country reference lookup for enrichment
Gold	`zomato.etl.zomato_summary`	Delta Lake	Analytics-ready, business-rule-applied output
Audit	`zomato.etl.zomato_summary_log`	Delta Lake	ETL execution metadata and step-level logging

1.5 Work Carried Out

The contribution of the internship to this project covered the complete lifecycle, from environment setup to module development, orchestration design, and validation. The work was structured into five phases:

Phase 1 – Environment Setup (Environment Setup Notebook):

The pipeline environment was initialized by creating the Unity Catalog hierarchy, provisioning the `zomato` catalog and `etl` schema, and setting up a managed volume (`zomato_files`). A structured folder hierarchy was created for raw inputs, staged files, CSV outputs, archives, and logs.

Delta tables (zomato_raw, dim_country, zomato_summary, and zomato_summary_log) were created with explicit schema definitions, partitioning strategies, and storage properties. The notebook was designed to be idempotent and safely re-executable.

Phase 2 – Module 1: JSON to CSV Conversion:

This module handles the ingestion stage. JSON files are dynamically identified in the raw landing zone, copied to a controlled processing path (source/json), and read using PySpark's multiline JSON reader.

The nested restaurant array is exploded and flattened into a tabular structure with 20 business-approved fields. A filedate column is derived from the filename or defaults to the current system date. The processed DataFrame is written as a single CSV file using coalesce(1) (e.g., zomato_1_<filedate>.csv). Processed JSON files are archived to avoid reprocessing.

Phase 3 – Module 2: Loading Raw Table and Dimension Table:

The generated CSV files are ingested into the zomato_raw Delta table with schema enforcement. An incremental load strategy is implemented: filedate values in incoming data are identified, corresponding partitions are deleted using targeted DELETE statements, and fresh data is inserted.

The dimension table (dim_country) is refreshed using a full overwrite strategy from a reference CSV, ensuring standardized column names aligned with the warehouse schema.

Phase 4 – Module 3: Summary Table Transformation and Loading:

This is the core transformation phase. The raw table is joined with the dimension table using a LEFT JOIN on country code to preserve all records.

Seven business rules are applied sequentially: (i) filtering records with NULL or blank cuisines; (ii) replacing NULL values in string columns; (iii) removing duplicate records based on restaurant_id and filedate; (iv) deriving m_rating_colour based on rating bands; (v) deriving m_cuisines using cuisine categorization logic; (vi) adding audit columns (create_datetime, user ID); (vii) renaming columns to partition keys (p_filedate, p_country_name).

The final DataFrame is appended to the zomato_summary table.

Phase 5 – Orchestration through Driver Notebook:

A centralized Driver Notebook orchestrates the entire pipeline. It includes runtime widgets to select load types (historical or manual), passes parameters to child notebooks using dbutils.notebook.run(), and logs execution status to the audit table.

A file-based locking mechanism prevents concurrent executions. Successful runs are archived, and logs are maintained with a 7-day retention policy using automated cleanup queries.

Table 2.2: Weekly Work Summary – Zomato ETL Project

Week	Activity	Outcome
1	Environment Setup and Architecture Design	Unity Catalog hierarchy, volume, folder structure, and Delta table schemas defined and validated
2	Module 1 Development (JSON $\rightarrow$ CSV)	JSON parsing, field extraction, filedate derivation, CSV generation, and archival logic implemented
3	Module 2 Development (Bronze Layer Load)	Incremental partition-level load strategy and dimension table refresh implemented and tested
4	Module 3 Development (Gold Layer Transformation)	Business rules applied; summary table created with partition keys and audit columns
5	Orchestration, Load Strategies, and Validation	Driver Notebook implemented; load strategies tested; end-to-end pipeline validated

1.6 Solution Provided

The solution provided is a modular, governance-first ETL pipeline built on Databricks that transforms raw Zomato restaurant data in JSON format into structured, auditable, and analytics-ready Delta Lake tables. The key engineering decisions and their business rationale are described below.

Unity Catalog as the Governance Backbone:

All pipeline resources—catalogs, schemas, tables, and volumes—are managed using Unity Catalog (`zomato.etl`). This ensures centralized access control, metadata management, and data discovery without relying on unmanaged storage paths. Each notebook explicitly defines the active catalog and schema to prevent unintended writes outside the governed namespace.

Medallion Architecture for Data Quality Progression:

The pipeline follows a three-layer Medallion Architecture. The Bronze layer (`zomato_raw`) stores minimally transformed, schema-aligned raw data, while the Gold layer (`zomato_summary`) contains fully processed, analytics-ready data. This separation allows safe reprocessing—transformations can be modified and reapplied without re-ingesting source data.

Incremental Load with Partition-Level Overwrite:

Instead of full table overwrites, the pipeline identifies specific `filedate` partitions in incoming data, deletes only those partitions from the Delta table, and inserts updated records. This approach is idempotent and computationally efficient, ensuring consistent results across repeated executions.

Computed Business Columns:

Two computed columns enhance analytical value. The `m_rating_colour` column

maps aggregate ratings into five bands (Red, Amber, Light Green, Green, Gold) using PySpark transformations. The `m_cuisines` column classifies restaurants into categories (e.g., Indian vs. World cuisines) based on keyword matching. These enrichments support intuitive filtering and segmentation in BI dashboards.

Audit Columns and Execution Logging:

Each record includes audit fields such as `create_datetime` and `user_id`, capturing execution context (system-generated, manual, or historical load). Additionally, pipeline execution metadata is stored in the `zomato_summary_log` table, including job ID, step name, timestamps, and execution status. This supports SLA monitoring, failure diagnosis, and compliance auditing.

Concurrency Safety via Lock Mechanism:

A file-based locking mechanism is implemented within the managed volume. The Driver Notebook creates a lock file at the start of execution and removes it in a `finally` block, ensuring release regardless of success or failure. If another execution detects an active lock, it aborts to prevent concurrent writes and data corruption.

Multiple Load Strategies:

The pipeline supports three execution modes: (i) *Scheduled load* for daily automated execution, (ii) *Manual load* using Databricks widgets to filter specific `filedate` or `country_code`, (iii) *Historical load* for full reprocessing of all available data.

This flexibility enables both steady-state operations and recovery/backfill scenarios.

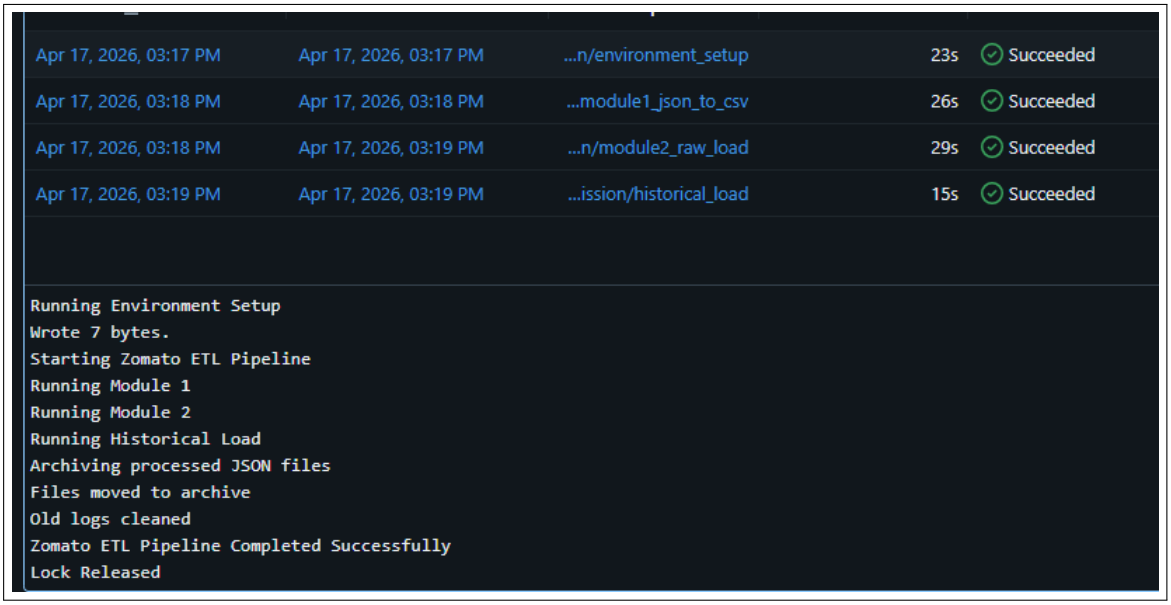


Figure 2.2: Driver Notebook – Step-wise Execution and Logging

The Driver Notebook executes each module sequentially, records the status of every step, and produces a Gold layer output that is country-enriched, rating-classified, audit-enabled, and partitioned for optimal query performance (Figure 2.2).

1.7 Results

The end-to-end testing of the pipeline was successfully completed across all three execution modes. Systematic validation confirmed the following outcomes:

The raw data was correctly ingested into the Bronze table `zomato_raw`, with proper partitioning by `filedate` and schema enforcement ensuring no mismatches during CSV ingestion. The dimension table `dim_country` was refreshed successfully, with country names accurately mapped to their corresponding codes.

All business rules were correctly applied in the `zomato_summary` (Gold) table. The final dataset contained no NULL values in the `cuisines` field, no duplicate combinations of `restaurant_id` and `filedate`, correctly derived rating color bands and cuisine classifications, and fully populated audit columns.

Partition pruning was validated successfully. Queries filtered by `p_country_name` or `p_filedate` scanned only the relevant Delta partitions, significantly reducing computational overhead for country-specific and date-specific analytics.

The concurrency control mechanism was tested by simulating parallel execution. The second execution attempt was correctly blocked, and the failure was recorded in the audit log table.

Table 2.3: Zomato ETL Pipeline Validation Test Cases

Test Case	Description	Expected Outcome	Status
TC-01	JSON parsing and 20-field extraction	All fields populated correctly	Pass
TC-02	Filedate derivation from filename	Correct date assigned per file	Pass
TC-03	NULL cuisine filtering	Zero NULL cuisine records in Gold table	Pass
TC-04	Deduplication on (`restaurant_id`, `filedate`)	No duplicate rows in summary	Pass
TC-05	Derivation of `m_rating_colour`	Correct rating band assignment	Pass
TC-06	Cuisine classification	Indian / World cuisines correctly assigned	Pass
TC-07	Incremental partition overwrite	Only target partitions refreshed	Pass
TC-08	Concurrency lock mechanism	Second run aborted and logged	Pass
TC-09	Manual load with filedate filter	Only specified records loaded	Pass
TC-10	Historical load (full overwrite)	Summary table rebuilt with schema intact	Pass

Table 2.4: Technology Stack – Zomato ETL Pipeline

Sr. No.	Technology / Tool	Role in Pipeline
1	Databricks Free Edition	ETL execution and notebook orchestration platform
2	Unity Catalog	Centralized governance, schema, and table management
3	Delta Lake	Storage layer for Bronze, Dimension, and Gold tables with ACID compliance
4	Databricks Volumes	Storage for raw, staged, CSV, and archived data
5	PySpark and Spark SQL	Data transformation, filtering, joining, and aggregation
6	Python (`dbutils`, `re`, `uuid`)	Orchestration utilities, filename parsing, and job tracking

1.8 Conclusion

The Zomato ETL Pipeline project presented an end-to-end, production-grade data engineering solution built on the Databricks Lakehouse platform. By implementing the Medallion Architecture, Unity Catalog for governance, Delta Lake for ACID-compliant storage, and a modular notebook-based orchestration framework, the pipeline effectively addressed key challenges related to data quality, governance, and operational reliability in processing semi-structured Zomato restaurant data.

From an engineering perspective, the project provided hands-on experience with real-world data platform design patterns, including partition-level incremental loading, schema enforcement, computed column generation, audit trail design, and concurrency control. These are fundamental skills required for building scalable and production-ready data systems.

From a business perspective, the resulting `zomato_summary` table enables analytics teams to perform country-level benchmarking, cuisine-based segmentation, rating distribution analysis, and pricing strategy evaluation with high confidence in data accuracy and completeness.

The multi-load strategy architecture (scheduled, manual, and historical) ensures operational flexibility and resilience. It supports not only routine daily processing but also targeted backfills and full data recovery scenarios without impacting downstream analytics.

Overall, this project established a strong foundation in modern data engineering practices and architectural design principles, which were further applied in subsequent retail and pharmaceutical data pipeline projects during the internship.

Project 2: RetailPulse Analytics Platform — End-to-End Retail Data Engineering on AWS

2.1 Introduction

Contemporary retail and e-commerce organizations generate massive volumes of transactional data every second, including customer purchases, inventory updates, partner fulfillment data, and payment events across multiple systems, formats, and geographies. Transforming this raw and unstructured data into reliable, analytics-ready datasets requires a scalable, governed, and automated cloud-native data platform.

Without such infrastructure, organizations face challenges such as inaccurate revenue reporting, inventory stockouts, delayed decision-making, and overall competitive disadvantage.

The RetailPulse Analytics Platform was developed during the internship at Infocepts as a comprehensive data engineering solution that simulates how modern retail enterprises design and operate end-to-end data platforms on Amazon Web Services (AWS). The platform ingests synthetic retail data from multiple sources, including relational databases (Amazon RDS MySQL), semi-structured JSON feeds from external partners, and flat-file inventory exports.

Data is processed using a distributed Apache Spark ETL pipeline on Amazon EMR, stored in a layered **Amazon S3 Data Lake**, and made available for serverless SQL analytics using **Amazon Athena** over the **AWS Glue Data Catalog**. Business intelligence is enabled through interactive dashboards built on Amazon QuickSight, while a real-time alerting system is implemented using AWS Lambda, Amazon SNS, and Amazon SQS.

The platform is designed based on five key principles: scalability, data quality, actionable analytics, multi-source integration, and operational automation. These principles reflect modern production-grade data platform architectures used in large-scale retail environments.

2.2 Problem Identification

The RetailPulse platform addresses several real-world data engineering challenges commonly observed in retail systems:

1. **Data Quality Degradation:** Raw transactional data often contains multiple quality issues. Duplicate records may occur due to repeated pipeline executions or source re-extractions, leading to inflated revenue metrics. Missing critical fields such as `order_id` and `customer_id` disrupt joins and aggregations. Future-dated transactions (orders with dates beyond the current date) distort time-series analysis and

reporting. Invalid pricing records, such as negative values or inconsistent discounts, result in incorrect margin calculations.

2. **Multi-Source Schema Heterogeneity:** Data is ingested from structurally different sources. Internal RDS tables use fields like `order_id`, `customer_id`, and `unit_price`, while external JSON feeds use `order_ref`, `cust_id`, and `price`. Without schema normalization, integrating these datasets into a unified analytical model becomes difficult.

3. **Lack of Real-Time Monitoring:** Traditional batch analytics systems do not provide mechanisms to detect critical business events in real time, such as stockouts of top-selling products, spikes in payment failures, or sudden drops in order volumes. This results in delayed operational responses.

4. **Absence of Cost-Optimized Querying:** Storing data in non-partitioned and non-columnar formats in Amazon S3 leads to full data scans during Athena queries. This increases both query execution time and cost, especially for large datasets.

5. **Lack of Referential Integrity Enforcement:** Orders referencing non-existent customer IDs (orphan records) are not detected or filtered. This leads to broken joins and inaccurate customer-level analytics.

2.3 Objectives

The RetailPulse Analytics Platform was designed with the following engineering and business objectives:

- Develop a **scalable, layered Amazon S3 Data Lake** with well-defined separation into Raw, Processed, Quarantine, and Aggregates zones to support data lineage and reprocessing flexibility.

- Build a multi-source ingestion pipeline that reads structured data from Amazon RDS (via JDBC) and semi-structured partner data from S3-hosted JSON files, and normalizes them into a unified schema using PySpark on Amazon EMR.

- Enforce data quality validation rules in Spark, including null key detection, future-date filtering, deduplication using window functions, and orphan customer detection. Invalid records are routed to structured quarantine zones in S3 for governance and investigation.

- Compute business metrics and enrich validated order data, including order revenue, discount percentage, profit margin, and weekend transaction flags, creating an analytics-ready fact table (`fact_orders`).

- Store processed data in partitioned Parquet format on S3 (partitioned by `order_date`) to enable efficient query pruning and cost-optimized analytics.
- Use AWS Glue Data Catalog with automated crawlers to manage schema metadata of processed datasets, enabling seamless querying through Amazon Athena without manual DDL management.
- Deliver business intelligence through Amazon QuickSight dashboards, including daily sales trends, product rankings, category revenue, order status distribution, and geographic revenue analysis using SPICE in-memory acceleration.
- Implement a serverless alerting pipeline using AWS Lambda, Amazon SNS, Amazon SQS, and Amazon RDS to detect and notify stakeholders of critical business anomalies such as stockouts, high cancellation rates, and zero-order categories.

2.4 Architecture Overview

The RetailPulse platform follows a cloud-native, multi-stage architecture on AWS, covering the complete lifecycle from data generation to real-time alerting. The high-level architecture is shown in Figure 2.3.

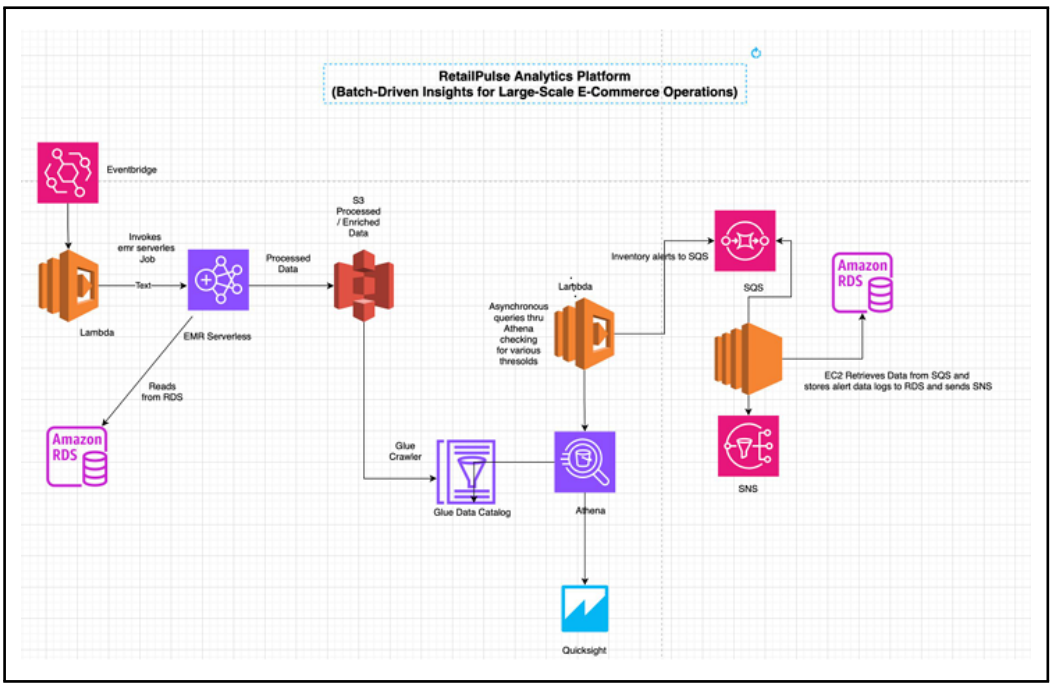


Figure 2.3: RetailPulse Analytics Platform End-to-End AWS Architecture

The architecture consists of the following stages:

Stage 1 – Data Generation: Synthetic retail datasets are generated using Python libraries such as Pandas, Faker, NumPy, and pytz. The datasets include customers, products,

distributors, orders (with intentional data quality issues), inventory snapshots, and external partner orders in JSON format.

Stage 2 – Amazon S3 Data Lake: The S3 bucket (`retailpulse-data-lake`) is structured into four zones: Raw (original files), Processed (clean Parquet datasets), Quarantine (rejected records such as null keys, duplicates, and invalid dates), and Aggregates (precomputed metrics).

Stage 3 – Amazon RDS (MySQL): Structured transactional data is stored in Amazon RDS and loaded using CSV files. It serves as a JDBC source for Spark-based ingestion.

Stage 4 – Amazon EMR (Apache Spark): The core ETL processing layer reads data from both RDS and S3, performs schema normalization, applies validation rules, computes business metrics, and writes enriched data in partitioned Parquet format to the Processed zone.

Stage 5 – AWS Glue and Amazon Athena: A Glue crawler automatically discovers schema and registers metadata in the Glue Data Catalog. Amazon Athena enables serverless SQL querying with partition pruning for efficient analytics.

Stage 6 – Amazon QuickSight: This layer provides interactive dashboards using Athena datasets with SPICE in-memory acceleration for fast query performance.

Stage 7 – Serverless Alerting: AWS Lambda, Amazon SNS, and Amazon SQS are used to implement automated monitoring and alerting workflows triggered via scheduled events.

Table 2.5: RetailPulse Analytics Platform – AWS Services Utilized

Stage	AWS Service / Tool	Role in Platform
1	Python (Pandas, Faker, NumPy, pytz)	Synthetic data generation with quality defect simulation
2	Amazon S3	Layered data lake (Raw, Processed, Quarantine, Aggregates)
3	Amazon RDS (MySQL)	Structured relational source; JDBC ingestion point
4	Amazon EMR (Apache Spark)	Distributed ETL processing: validation, transformation, enrichment
5	AWS Glue Data Catalog	Automated schema discovery and metadata management
6	Amazon Athena	Serverless SQL analytics with partition pruning on Parquet data
7	Amazon QuickSight (SPICE)	Interactive BI dashboards for business users
8	AWS Lambda, SNS, SQS	Serverless alerting, notification, and message queuing

2.5 Work Carried Out

The internship contribution spanned the complete lifecycle of the platform across seven stages. Each stage was built incrementally on the outputs of the previous stage.

Stage 1 – Synthetic Data Generation:

Python scripts were used to generate six datasets. The customer dataset (1,000 records) included demographic attributes such as city, state, pincode, customer segment, loyalty tier, acquisition channel, and date of birth using the Faker library (en_IN locale) to simulate the Indian retail environment.

The product catalog (200 products) included category, sub-category, brand, MRP, cost price, and reorder thresholds. The orders dataset (5,000+ records) was generated with intentional data quality issues: approximately 5% NULL values, 3% future-dated transactions, and 4% duplicate records. Additional datasets included distributors, inventory snapshots, and partner JSON feeds. All datasets were exported in CSV/JSON format using Pandas.

Stage 2 – Amazon S3 Data Lake Setup:

An S3 bucket (`retailpulse-data-lake`) was created in the `ap-south-1` region. A four-zone architecture was implemented: `raw/` (landing zone), `processed/` (clean Parquet data), `quarantine/` (invalid records categorized by type), and `aggregates/` (precomputed metrics).

Data was uploaded into the raw zone using AWS CLI commands (`aws s3 cp`).

Stage 3 – Amazon RDS Configuration and Schema Loading:

A MySQL 8.x RDS instance (`retailpulse-db`) was provisioned with appropriate VPC security groups allowing EMR connectivity. Five tables (`customers`, `orders`, `products`, `distributors`, `inventory`) were created with proper DDL schemas, constraints, and primary keys.

Data was loaded using the `LOAD DATA LOCAL INFILE` command after transferring files via an EC2 bastion host.

Stage 4 – EMR Spark ETL Pipeline:

A PySpark job was executed on an EMR cluster. Spark was configured with `spark.sql.shuffl` for optimal parallelism.

Data from RDS was read via JDBC, and partner JSON data was read from S3. Schema normalization was performed by aligning column names (e.g., `order_ref` to `order_id`, `cust_id` to `customer_id`). The datasets were merged using `unionByName(allowMissingCo`

Four validation rules were applied: (i) NULL key detection using `isNull()`, (ii) future-date filtering using `current_timestamp()`, (iii) deduplication using window functions, (iv) orphan customer detection via left join.

Invalid records were written to respective quarantine zones in Parquet format.

The dataset was enriched with: `revenue = quantity * unit_price`, `discount_pct = (mrp - unit_price) / mrp * 100`, `is_weekend` using `dayofweek()`, and profit margin calculations.

The final dataset was written to `processed/facts/orders/` partitioned by `order_date`.

Stage 5 – AWS Glue and Athena Analytics:

A Glue crawler was configured to scan the processed data, infer schema, and register the `fact_orders` table in the Glue Data Catalog.

Athena queries implemented included: daily revenue trends with `LAG()`, top product ranking using `RANK()`, customer cohort analysis, and inventory risk analysis.

Stage 6 – Amazon QuickSight Dashboards:

QuickSight was connected to Athena via the Glue Catalog. Five dashboards were created using SPICE: Daily Sales Summary, Top Products, Category Revenue, Order Status Distribution, and Geographic Revenue Map.

Stage 7 – Lambda Alerting System:

An SQS queue (`retailpulse-alert-queue`) and SNS topic (`retailpulse-alert-top`) were created. A Lambda function (Python, boto3) executed Athena queries, evaluated business conditions, and triggered alerts.

Alerts were published to SNS and SQS, and stored in an RDS table (`alerts_log`) for audit purposes. The system was scheduled using CloudWatch Events (every 5 minutes) for near real-time monitoring.

Table 2.6: Weekly Work Summary – RetailPulse Analytics Platform

Week	Activity	Outcome
1	Data Generation and S3 Setup	Synthetic datasets created; S3 four-zone architecture implemented
2	RDS Schema Design and Loading	MySQL tables created; data loaded; JDBC connectivity validated
3	EMR Spark ETL Development	Validation rules applied; data unified; quarantine zones populated
4	Glue Catalog and Athena Analytics	Schema registered; SQL queries validated; dashboards created
5	Lambda Alerting and Validation	Alert system deployed; end-to-end pipeline validated

2.6 Solution Provided

The RetailPulse Analytics Platform delivers a production-grade AWS data engineering solution. The following key engineering decisions define the solution and its business value.

Layered S3 Data Lake with Typed Quarantine Zones:

Instead of a simple accept/reject mechanism, invalid records are routed to semantically organized quarantine subfolders such as `null_key/`, `future_date/`, and `duplicates/`, stored in Parquet format. This enables systematic analysis of data quality issues, tracking defect trends over time, and improving upstream systems. The raw zone remains immutable, ensuring a tamper-proof audit trail and reprocessing capability.

Schema Normalization using Column Renaming and `unionByName()`:

Partner JSON data uses different field names compared to internal RDS schemas. To address schema heterogeneity, columns are explicitly renamed before merging datasets. The use of `unionByName(allowMissingColumns=True)` ensures compatibility even when optional fields are absent, making the pipeline robust to schema evolution.

Window Function-Based Deduplication:

Duplicate records are identified using window functions by partitioning data on `order_id` and ordering by `order_date`. Only the first occurrence is retained. This deterministic approach ensures consistent results regardless of data distribution and supports parallel execution in Spark.

Parquet Storage with Date Partitioning for Cost Optimization:

Processed data is stored in columnar Parquet format and partitioned by `order_date`. Amazon Athena leverages partition pruning to scan only relevant data, significantly reducing query cost and execution time. For example, querying a 7-day window from a 365-day dataset reduces data scan by over 95%.

Business Enrichment Measures:

The pipeline computes key analytical fields:

- `revenue` = quantity $\times$ unit_price
- `discount_pct` = (mrp - unit_price) / mrp $\times$ 100
- `is_weekend` using `dayofweek()`
- `profit_margin` = revenue - cost_price

These enrichments enable direct analytical consumption without requiring additional joins.

Advanced Athena Analytics:

Athena queries use SQL window functions such as `LAG()` for day-over-day revenue growth and `RANK()` for product ranking. Conditional logic using `CASE` expressions enables inventory risk classification (e.g., AT RISK vs STABLE), which feeds directly into alerting workflows.

Fault-Tolerant Serverless Alerting:

The Lambda–SNS–SQS architecture ensures reliable alert delivery. AWS Lambda executes monitoring queries, SNS provides real-time notifications, and SQS ensures message

durability and retry capability. Alerts are stored in the `alerts_log` table in RDS for auditing and trend analysis.

Four key alert types are implemented:

- Top-selling product stockouts
- High cancellation rates (e.g., $>25\%$)
- Zero-order category detection
- Sudden drop in daily revenue

As illustrated in Figure 2.4, the Spark ETL layer processes data through sequential validation stages before enrichment, while the QuickSight layer presents clean, analytics-ready insights through interactive dashboards.

2.7 Results

The platform was tested end-to-end across all seven stages. Systematic validation and query-level verification confirmed the following outcomes.

Data was correctly partitioned across all four zones of the S3 Data Lake. The quarantine layer successfully captured invalid records, including NULL key records, future-dated transactions, and duplicates. These records were written to their respective subfolders in Parquet format and validated using Athena queries.

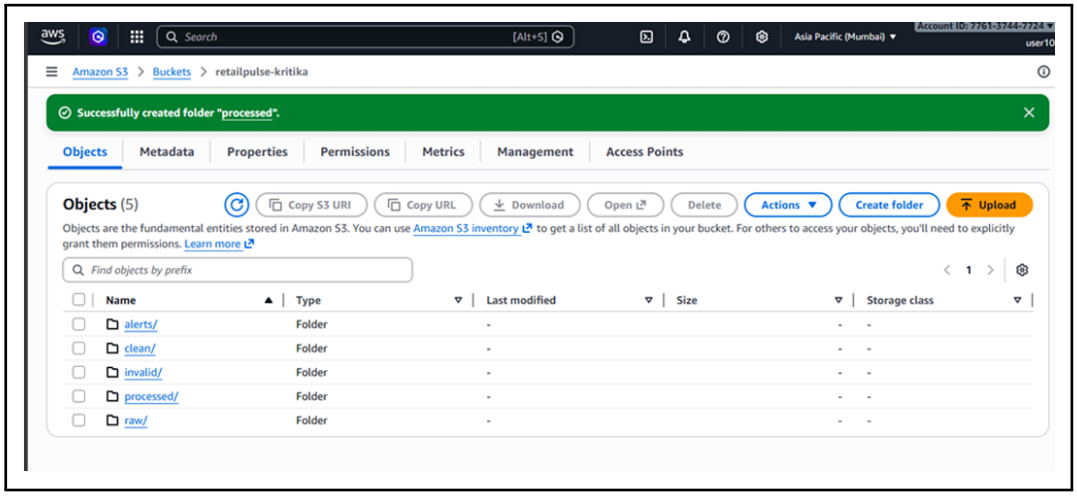
The processed `fact_orders` table contained zero NULL critical keys, zero future-dated records, and no duplicate `order_id` entries. All enrichment columns (`revenue`, `discount_pct`, `is_weekend`, `profit_margin`) were correctly computed and stored.

Partition pruning was validated successfully. A 7-day Athena query on a 90-day dataset scanned only the required partitions, significantly reducing data scan volume and query cost. Advanced Athena queries produced accurate results, including top product rankings, revenue trends, customer cohort analysis, and inventory risk detection.

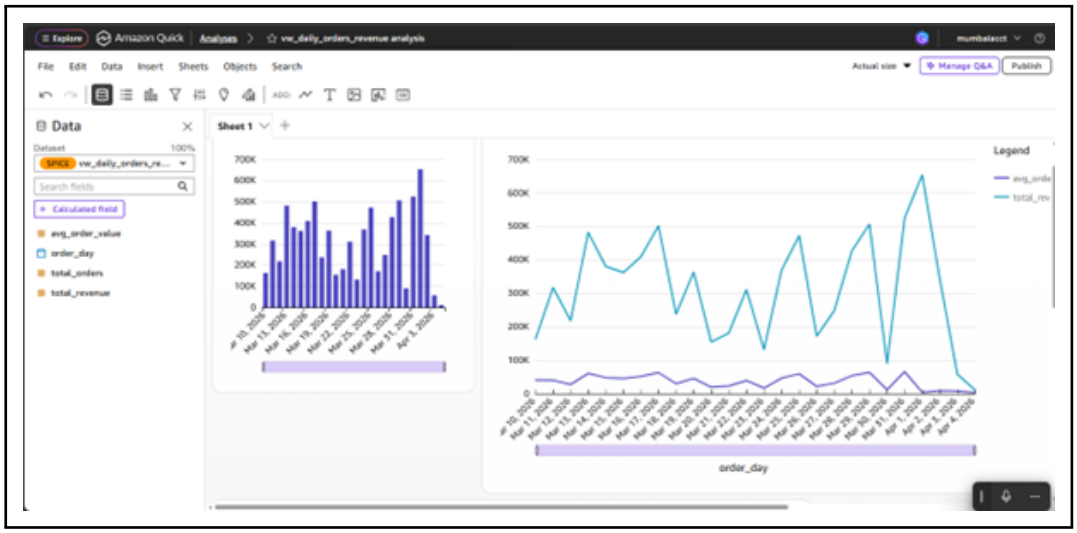
QuickSight dashboards loaded successfully using SPICE, and all five dashboards displayed accurate and interactive visualizations. The Lambda alerting system correctly detected injected anomalies, triggered SNS notifications, queued messages in SQS, and logged alerts in the RDS `alerts_log` table.

2.8 Conclusion

The RetailPulse Analytics Platform successfully delivers a complete, production-grade, cloud-native data engineering solution on AWS. It addresses key challenges in data quality, multi-source integration, scalable storage, cost-efficient analytics, business intelligence delivery, and automated monitoring within a retail environment.



(a) S3 Data Lake – Validation and Quarantine Routing



(b) QuickSight Dashboards – Sales Summary and Category Revenue

Figure 2.4: RetailPulse Platform – Spark Processing and BI Output

Table 2.7: RetailPulse Analytics Platform – Validation Test Cases

Test Case	Description	Expected Outcome	Status
TC-01	NULL `order_id` / `customer_id` detection	Records routed to `quarantine/null_key/`	Pass
TC-02	Future-dated order filtering	Records routed to `quarantine/future_date/`	Pass
TC-03	Duplicate order deduplication	Only first occurrence retained	Pass
TC-04	Orphan customer detection	Missing customers flagged, not dropped	Pass
TC-05	Schema normalization (JSON + RDS)	Unified schema correctly applied	Pass
TC-06	Revenue and discount calculations	All computed values correct	Pass
TC-07	Partition pruning in Athena	Only required partitions scanned	Pass
TC-08	Glue crawler schema inference	`fact_orders` registered correctly	Pass
TC-09	QuickSight SPICE ingestion	Dashboards rendered successfully	Pass
TC-10	Lambda alert (high cancellation)	SNS, SQS, and RDS logging successful	Pass
TC-11	Lambda alert (top-seller stockout)	Alert triggered correctly	Pass
TC-12	Lambda alert (zero-order category)	Alert generated correctly	Pass

From an engineering perspective, the project provided extensive hands-on experience with AWS services such as EMR, RDS, S3, Glue, Athena, QuickSight, Lambda, SNS, and SQS. It also reinforced core data engineering concepts including schema normalization, window-function-based deduplication, quarantine-based data quality management, and partitioned columnar storage optimization.

From a business perspective, the platform enables a shift from reactive reporting to proactive, data-driven decision-making. QuickSight dashboards provide actionable insights into sales trends, product performance, and customer behavior. The real-time alerting system ensures that critical operational events—such as stockouts, high cancellation rates, and demand anomalies—are detected and communicated within minutes.

The architecture is highly scalable and adaptable. By adjusting EMR cluster configurations, Spark partitioning, and Glue crawler schedules, the platform can handle significant increases in data volume without structural changes.

Overall, this project demonstrates the design and implementation of a modern, scal-

able, and intelligent retail data platform aligned with real-world enterprise data engineering practices.

Project 3: Pharmaceutical Supply Chain Streaming Analytics Platform

3.1 Introduction

Pharmaceutical supply chains operate under strict regulatory, quality, and operational constraints that traditional data platforms are not designed to handle. Temperature-sensitive medicines such as vaccines, insulin, biologics, and controlled substances require continuous cold chain monitoring from manufacturing through storage, distribution, and dispensing.

Each manufactured batch must have complete traceability, linking manufacturing records, quality control test results, distributor inventory movements, and pharmacy sales transactions. In the event of a post-release quality failure, organizations must rapidly assess recall impact by identifying affected orders, in-transit inventory, and revenue exposure within hours rather than days.

Conventional pharmaceutical data systems fail to meet these requirements due to their siloed and batch-oriented nature. Temperature excursions are often detected with delays, impact assessments rely on manual processes, inventory visibility is fragmented across distributors, and quality control data is not integrated with real-time operational data.

This project, conducted during the internship at Infocepts, presents an end-to-end, event-driven pharmaceutical supply chain analytics platform. The platform integrates real-time streaming, batch processing, and unified data modeling to address these challenges.

Real-time data such as temperature sensor readings, pharmacy sales, and distributor inventory updates are ingested using **Amazon Kinesis Data Streams**. These streams are processed in real time using Spark Structured Streaming on Amazon EMR and stored in an S3 data lake managed through Hive.

Batch data, including product master, batch master, inventory snapshots, and QC test results, is ingested from Amazon RDS using Apache Sqoop and stored in Hive tables across multiple layers (raw, clean, valid, and invalid). A unified data model is achieved using a shared `batch_id` as the central linking key across all datasets.

The final analytical layer is powered by **Amazon Athena** and visualized using QuickSight dashboards. This enables stakeholders to monitor cold chain compliance, track inventory risk, ensure batch traceability, and assess recall impact in near real time.

3.2 Problem Identification

The pharmaceutical supply chain analytics platform was designed to address four critical business and engineering challenges:

1. **Lack of Real-Time Cold Chain Monitoring:** Temperature-sensitive pharmaceutical products must be stored and transported within strict temperature ranges (e.g., 2–8°C). IoT sensors generate temperature readings at frequent intervals (e.g., every 30 seconds). Without real-time streaming systems, temperature excursions are detected late through manual audits, often after products have already moved through the supply chain. This creates compliance risks and potential harm to patients.

2. **Lack of Batch-Level End-to-End Traceability:** Every dispensed unit must be traceable back to its manufacturing batch, including associated QC results, temperature history, and supply chain movements. However, data is distributed across multiple systems such as ERP (manufacturing), LIMS (quality), WMS (distribution), and POS (pharmacy). The absence of a unified linking mechanism makes end-to-end traceability difficult. The `batch_id` serves as the natural key to integrate these datasets.

3. **Fragmented Inventory Visibility:** Distributor and warehouse inventory data is siloed, preventing real-time visibility into stock levels. Without continuous monitoring of stock coverage and demand patterns, replenishment decisions become reactive, leading to stockouts or excess inventory nearing expiry.

4. **Delayed Quality Failure Impact Assessment:** When a batch fails quality testing (e.g., abnormal potency, sterility issues, endotoxin levels), identifying affected orders, inventory locations, and revenue exposure is a manual and time-consuming process. This delay (often 24–72 hours) increases the risk of defective products reaching patients.

3.3 Objectives

The following are the engineering and compliance goals of the Pharmaceutical Supply Chain Analytics Platform:

- Use a real-time streaming ingestion pipeline with Amazon Kinesis Data Streams and Spark Structured Streaming on Amazon EMR to continuously ingest temperature sensor events, pharmacy sales transactions, and distributor inventory events with latency under one minute.

- Implement stream-level data validation including temperature unit conversion (Fahrenheit to Celsius), range validation (-50°C to 100°C), positive sales quantity and price checks, and inventory change type validation. Valid records are stored while invalid events are discarded.
- Build a batch ETL pipeline using Apache Sqoop to extract manufacturing batch master, product master, QC results, and inventory snapshots from Amazon RDS MySQL into Hive.
- Create a unified data spine using `batch_id` across all datasets to enable end-to-end traceability across temperature, sales, inventory, and QC data.
- Enable near real-time recall impact analysis by linking QC failures with live orders and inventory data.
- Execute six Athena analytical queries covering cold chain compliance, stockout risk, recall impact, distributor violations, abnormal sales patterns, and QC adherence.
- Build QuickSight dashboards using SPICE for cold chain monitoring, inventory risk, and operational insights.

3.4 Architecture Overview

The platform is designed with two parallel processing lanes — a real-time streaming lane and a batch processing lane — which converge at the Hive metastore and are queried using Amazon Athena and visualized via QuickSight.

Streaming Lane:

Three event streams — temperature sensor readings, pharmacy sales transactions, and distributor inventory updates — are generated using a Python script (`producer.py`) running on an EC2 instance.

The producer reads CSV files, assigns a `source_type`, shuffles records, and publishes them to a Kinesis Data Stream (`pharma_kinesis_kritika`) at 30-second intervals using `boto3`.

A PySpark Structured Streaming application (`consumer.py`) running on EMR:

- Reads data from Kinesis using the `spark-sql-kinesis` connector
- Parses JSON using `from_json()`
- Splits data into three streams using `source_type`
- Applies validation and transformations

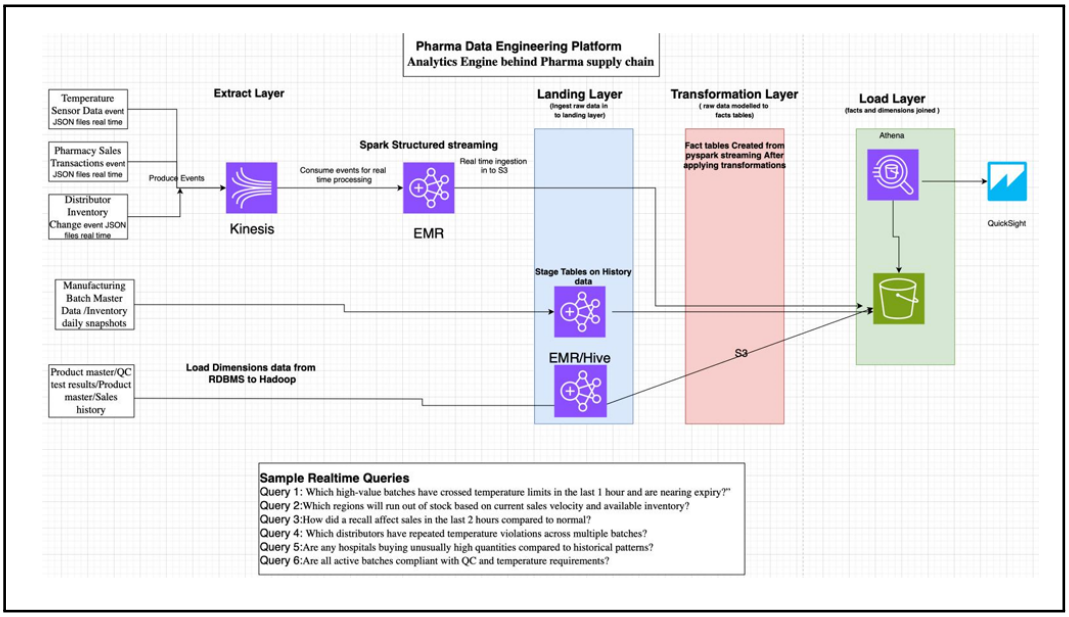


Figure 2.5: Pharmaceutical Supply Chain Analytics Platform – Kinesis + Spark Architecture

- Writes output to Hive-backed Parquet tables in S3

Micro-batch interval: 120 seconds with checkpointing.

Batch Lane:

Four datasets — batch master, product master, QC results, and inventory snapshots — are extracted from Amazon RDS MySQL using `sqoop import`.

In Hive, each dataset is processed through:

- `_raw` – raw ingestion
- `_clean` – type casting and cleaning
- `_valid` – business-rule validated data
- `_invalid` – rejected data

Validated data is exported to S3 in Parquet format for Athena querying.

Analytical Layer:

Amazon Athena queries the S3 Parquet datasets using Glue Catalog (`pharma_db_kritika`). Six analytical queries join batch and streaming data using `batch_id`.

QuickSight consumes these datasets via SPICE to create dashboards.

3.5 Work Carried Out

The internship contribution involved the complete implementation of both streaming and batch pipeline lanes, along with Athena analytics and QuickSight visualization.

Table 2.8: Data Sources and Schemas – Pharmaceutical Supply Chain Platform

Source	Dataset	Transport	Key Fields
Streaming	Temperature Sensor Events	Kinesis (30 sec interval)	`sensor_id, batch_id, temperature_c, alert_flag`
Streaming	Pharmacy Sales Transactions	Kinesis (real-time)	`transaction_id, batch_id, revenue_inr, fulfillment_status`
Streaming	Distributor Inventory Changes	Kinesis (real-time)	`event_id, batch_id, change_type, current_stock`
Batch	Manufacturing Batch Master	S3 (daily)	`batch_id, min_temp_c, max_temp_c, batch_status`
Batch	Product Master	RDS (daily)	`product_sku, cold_chain_required, controlled_flag`
Batch	QC Test Results	LIMS (daily incremental)	`qc_id, batch_id, overall_result, potency_pct`
Batch	Inventory Snapshot	WMS (daily)	`snapshot_date, distributor_id, closing_stock, reorder_point`

Phase 1 – Kinesis Stream Provisioning and EC2 Producer Setup:

An Amazon Kinesis Data Stream (`pharma_kinesis_kritika`) was provisioned in on-demand mode in the `ap-south-1` region to handle variable throughput without manual sharding.

An EC2 instance (t3.micro, Amazon Linux 2023) was launched and configured with Python 3, `boto3`, and `pandas` using `pip install`. AWS CLI credentials were configured for secure access.

Three datasets — `temperature_events.csv`, `sales_transactions.csv`, and `inventory_events.csv` — were uploaded via SCP.

The `producer.py` script:

- Reads all datasets
- Adds a `source_type` field
- Merges and shuffles records

- Publishes records to Kinesis using `kinesis.put_record()`
- Uses random partition keys and `time.sleep(30)` delay

Streaming was validated using terminal logs indicating successful record transmission.

Phase 2 – EMR Spark Structured Streaming Consumer:

The Kinesis connector JAR (`spark-sql-kinesis_2.12-1.2.0_spark-3.0.jar`) was uploaded to the EMR cluster.

The `consumer.py` application:

- Initializes SparkSession with Hive support
- Reads Kinesis stream using: `spark.readStream.format("aws-kinesis")`
- Converts binary data to string: `CAST(data AS STRING)`
- Parses JSON using `from_json()`
- Splits streams using `source_type`

Stream-specific transformations:

- **Temperature Stream:** Converts Fahrenheit to Celsius, validates range (-50°C to 100°C)
- **Sales Stream:** Casts quantity and price, filters invalid values
- **Inventory Stream:** Validates `change_type` against whitelist

A partition column `partition_date` was derived using `to_date()`.

Data was written to Hive-managed Parquet tables:

- `pharma_streaming_db.temperature_streaming`
- `pharma_streaming_db.sales_streaming`
- `pharma_streaming_db.inventory_streaming`

Streaming jobs used:

- Micro-batch interval: 120 seconds
- S3-based checkpointing

Execution:

```
spark-submit --jars spark-sql-kinesis_2.12-1.2.0_spark-3.0.jar consu
```

Validation was performed using Spark SQL queries.

Phase 3 – Batch Pipeline: Sqoop Extraction and Hive Processing:

MySQL Connector/J was added to Sqoop libraries.

Sqoop imports:

- Extracted `product_master` and `qc_results`
- Used JDBC connection to Amazon RDS
- Stored data in HDFS as text files

Hive processing included four layers:

- `_raw`
- `_clean`
- `_valid`
- `_invalid`

Data cleaning techniques:

- `CAST()`, `TRIM()`, `NULLIF()`
- Standardization using `UPPER()`
- Validation using `CASE` conditions

Example validation:

- `overall_result IN ('PASS','FAIL')`
- `potency_pct BETWEEN 80 AND 120`

Results:

- Total QC records: 200
- Valid: 183
- Invalid: 17

Valid data was exported to S3 in Parquet format:

`s3://pharma-data-lake-kritika/hive_exports/pharma_db_kritika/`

Phase 4 – Athena Analytics and QuickSight Dashboards:
Athena was configured using Glue Data Catalog (`pharma_db_kritika`).
Six analytical queries were implemented:

- Cold chain compliance violations
- Stockout risk detection
- Recall impact analysis
- Distributor compliance tracking
- Abnormal sales pattern detection
- Batch compliance evaluation

QuickSight integration:

- Connected via Athena data source
- SPICE datasets created
- Dashboards built:
  - Cold Chain Compliance Analysis
  - Regional Sales Performance

Table 2.9: Weekly Work Summary – Pharmaceutical Supply Chain Streaming Platform

Week	Activity	Outcome
1	Kinesis Provisioning & EC2 Producer Development	Stream created; producer publishing events with 30-second intervals; validated via logs
2	EMR Spark Streaming Consumer	Validated and stored three streams into Hive tables with checkpointing
3	Sqoop Extraction & Hive Processing	Batch data ingested; four-layer Hive model implemented
4	S3 Export, Athena Analytics & QuickSight	Parquet export completed; Athena queries executed; dashboards published

3.6 Solution Provided

The platform provides four tightly integrated technical capabilities that together solve key pharmaceutical supply chain challenges.

Real-Time Cold Chain Tracking using Kinesis and Spark Structured Streaming:

IoT temperature readings are streamed every 30 seconds to Amazon Kinesis and processed in 120-second micro-batches using Spark Structured Streaming.

Temperature normalization is performed using:

$$T_C = (T_F - 32) \times \frac{5}{9}$$

A validation filter ensures values lie within the physical range (-50°C to 100°C). An `alert_flag` is triggered when temperature violates batch-specific thresholds (`min_temp_c`, `max_temp_c`) from the batch master.

This enables near real-time (¡3 minutes) cold chain breach detection.

Unified Batch-Centric Data Spine using `batch_id`:

All datasets are linked using the common key `batch_id`. The analytical layer joins:

- `temp_valid` with `batch_valid` for compliance tracking
- `sales_valid` with `batch_valid` for transaction linkage
- `inventory_valid` with batch data for stock analysis

This enables complete end-to-end traceability from manufacturing to dispensing.

Four-Tier Hive Architecture for Batch Processing:

Each dataset follows four layers:

- `_raw` – original HDFS data (immutable)
- `_clean` – type casting, trimming, null handling
- `_valid` – business-rule filtered data
- `_invalid` – rejected records

Example validation rules:

- `potency_pct BETWEEN 80 AND 120`
- `overall_result IN ('PASS','FAIL')`

Out of 200 QC records:

- Valid: 183 (91.5%)
- Invalid: 17

Automated Recall Impact Analysis:

Athena queries join `sales_valid` with `batch_valid` where:

```
batch_status = 'RECALLED'
```

Revenue during recall is compared against historical baselines, enabling rapid impact assessment within seconds instead of hours.

3.7 Results

The platform was successfully validated across both streaming and batch processing pipelines.

Streaming Results:

- Kinesis producer generated interleaved temperature, sales, and inventory events
- Spark consumer correctly split streams and applied validations
- Data written to Hive tables:
  - `temperature_streaming`
  - `sales_streaming`
  - `inventory_streaming`
- Partitioned Parquet files stored in S3: `streaming_kinesis/temperature/`, `sales/`, `inventory/`

Batch Results:

- Sqoop successfully extracted data from RDS to HDFS
- Hive pipeline produced:
  - 200 total QC records
  - 183 valid records
  - 17 invalid records
- All datasets exported to S3 in Parquet format
- Athena tables created and queried successfully

Analytics and Dashboard Results:

- Athena queries executed successfully for all use cases
- QuickSight datasets (8,400 rows) imported with 100% success
- Dashboards displayed correctly without errors

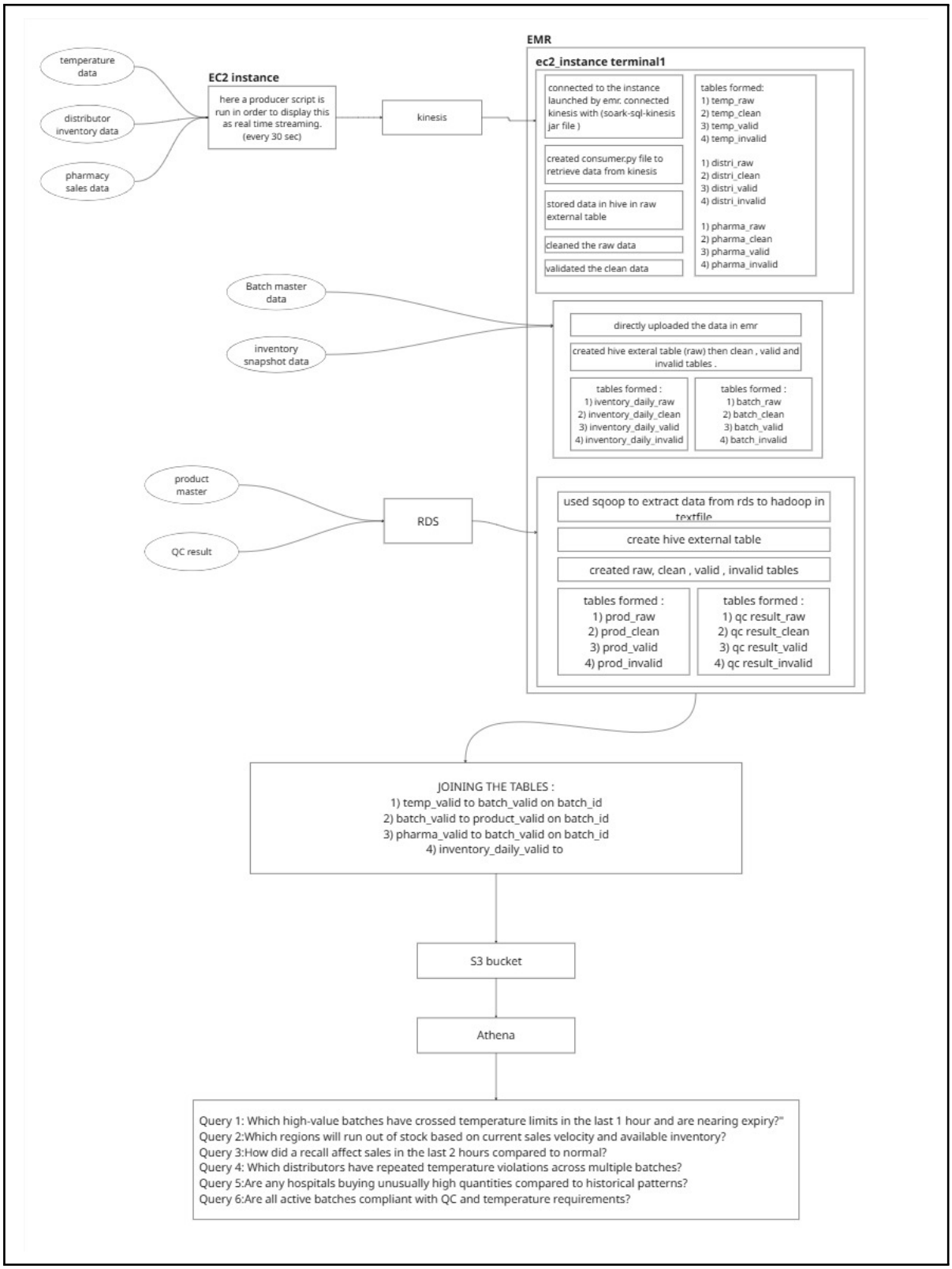


Figure 2.6: Detailed Architecture of Pharmaceutical Supply Chain Platform

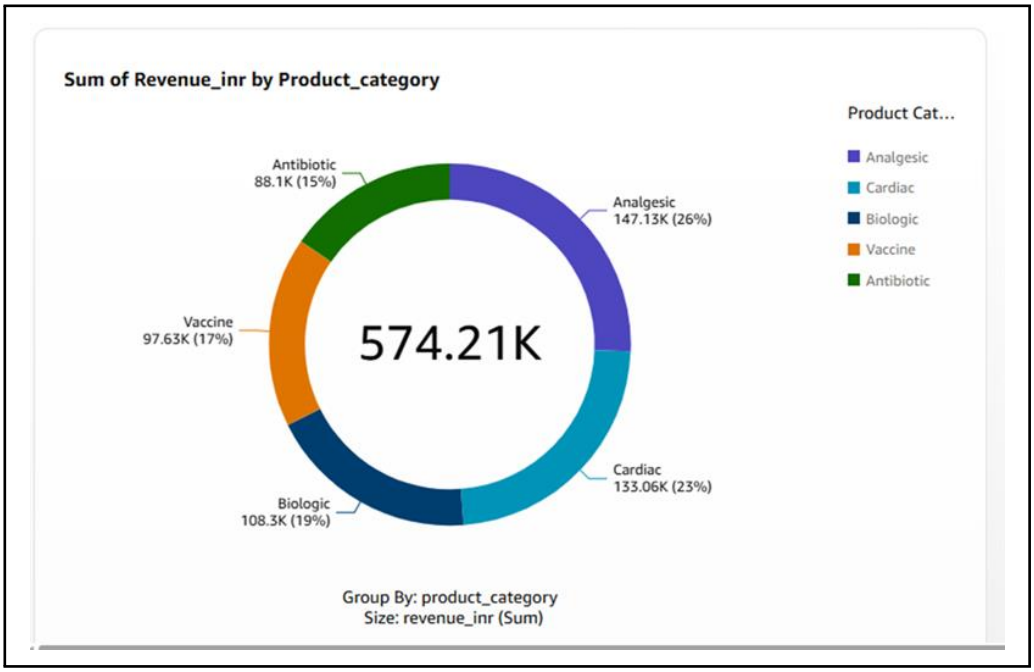


Figure 2.7: QuickSight Dashboard – Cold Chain Compliance and Regional Sales Analysis

3.8 Conclusion

The Pharmaceutical Supply Chain Streaming Analytics Platform project successfully delivered a production-grade, dual-lane data engineering system that transforms pharmaceutical operations from reactive, batch-refreshed reporting to near-real-time decision intelligence. By integrating Amazon Kinesis Data Streams, Spark Structured Streaming, Amazon EMR with Hive, Apache Sqoop, Amazon Athena, and Amazon QuickSight into a coherent, `batch_id`-linked analytical platform, the system addresses all four identified pharmaceutical supply chain challenges — cold chain monitoring, batch traceability, inventory visibility, and recall impact assessment — within a single, unified data architecture.

From an engineering standpoint, this project was the most technically advanced of the three internship engagements, requiring proficiency in real-time event streaming, Kinesis producer and consumer design, Spark Structured Streaming with Hive integration, JDBC-based batch extraction via Sqoop, multi-tier Hive ETL with complex business rule validation, cross-domain analytical query design in Athena, and cloud-native BI delivery via QuickSight. The combination of streaming and batch processing within a single cohesive platform reflects the Lambda Architecture pattern adopted by enterprise-grade pharmaceutical data platforms.

From a business and compliance standpoint, the platform delivers measurable impact across four dimensions. Cold chain breach detection latency is reduced from several hours to under three minutes, enabling regulatory-grade continuous compliance evidence. End-

Table 2.10: Pipeline Validation Test Cases – Pharmaceutical Supply Chain Platform

Test Case	Description	Expected Outcome	Status
TC-01	Kinesis producer emission	Records from all sources published	Pass
TC-02	Temperature conversion	Correct Celsius values computed	Pass
TC-03	Temperature validation	Out-of-range values filtered	Pass
TC-04	Sales validation	Invalid quantity/price removed	Pass
TC-05	Inventory validation	Valid change types retained	Pass
TC-06	Streaming writes	Data persisted with checkpoints	Pass
TC-07	Sqoop extraction	Data successfully loaded to HDFS	Pass
TC-08	Hive validation pipeline	Correct valid/invalid split	Pass
TC-09	Product validation	Invalid temperature ranges rejected	Pass
TC-10	S3 Parquet export of valid batch tables	All 12 table variants confirmed in S3 listing	Pass
TC-11	Athena cross-domain join on `batch_id`	Streaming and batch data correctly joined	Pass
TC-12	QuickSight SPICE ingestion (both datasets)	8,400 rows; 100% success; 0 rows skipped	Pass

to-end batch traceability is established through a single `batch_id` linkage across seven datasets. Recall impact assessment time is reduced from 24–72 hours to seconds via Athena cross-domain joins. Proactive inventory replenishment decisions become possible through real-time days-of-stock coverage computation. Together, these capabilities represent a significant reduction in compliance risk, patient safety exposure, and financial uncertainty for pharmaceutical supply chain operations.

BIBLIOGRAPHY

A. PLAGIARISM REPORT

Breast_Cancer_SL.docx

ORIGINALITY REPORT

9%	3%	7%	3%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Tulane University Student Paper	1%
2	Sumit Garethiya, Himanshu Agrawal, Shilpa Gite, Suresh. V, Amit Kudale, Gaurav Wable, Girishchandra R. Yendargaye. "Affordable system for alerting, monitoring and controlling heat stroke inside vehicles", 2015 International Conference on Industrial Instrumentation and Control (ICIC), 2015 Publication	1%
3	Abdul, Abdullah. "Using Machine Learning Algorithms to Find Novel Biomarkers for Breast Cancer Using RNA-Seq Dataset", Morgan State University, 2023 Publication	1%
4	H L Gururaj, Francesco Flammini, V Ravi Kumar, N S Prema. "Recent Trends in Healthcare Innovation", CRC Press, 2025 Publication	1%
5	Mahmoud Darwich, Magdy Bayoumi. "An evaluation of the effectiveness of machine learning prediction models in assessing breast cancer risk", Informatics in Medicine Unlocked, 2024 Publication	1%
6	doctorpenguin.com Internet Source	1%
7	scienceofbiogenetics.com Internet Source	1%
	Submitted to Nottingham Trent University	

A Machine Learning-Based Predictive Model for Early Detection of Breast Cancer

1st Aaryan Tamhane
Computer Science
Symbiosis Institute of Technology
Nagpur India
aaryantamhane29@gmail.com

2nd Advait Kulkarni
Computer Science
Symbiosis Institute of Technology
Nagpur India
kulkarniadvait93@gmail.com

3rd Soham Shrawankar
Computer Science
Symbiosis Institute of Technology
Nagpur India
sohamshrawankar@gmail.com

4th Sagarkumar Badhiye
Computer Science
Symbiosis Institute of Technology
Nagpur India
sagarkumar.badhiyc@sitnagpur.siu.edu.in

5th Pradnya Borkar
Computer Science
Symbiosis Institute of Technology
Nagpur India
pradnyaborkar2@gmail.com

***Abstract*—Breast cancer is one of the biggest health risks today, and early diagnosis is essential for a better prognosis of the disease. Data-driven approaches such as machine learning emerge successful in diagnosis and breast cancer prediction. This section assesses several general classifiers for prediction such as Random forest (RF), Logistic Regressions (LR) etc. algorithms explored on breast cancer datasets. The dataset is a fine needle aspiration (Biopsy) dataset namely Wisconsin Breast Cancer dataset, which was obtained from the UCI Machine Learning (ML) database. Discussions on the comparisons of different algorithms, feature selection methods, and performance measures are also covered.**

***Index Terms*—Breast-cancer, Machine-learning, Classification algorithms, SVM, Random forest, Feature selection, Data mining.**

I. Introduction

Breast-cancer is a serious and common type of cancer that is a great cause of death among women worldwide [1]. World Health Organization specifies it as a leading cause of death, responsible for around the fifteenth percent of all cancer death rates in women [5]. Timely and accurate detection followed by judicious handling of the disease path in the first stages, improves prognosis and survival rates- vastly [1]. While the accuracy, consistency, and efficiency of traditional diagnostic methods, such as mammography, are problematic in the early stages of the disease, non-invasive methods powered by modern technologies usually demonstrate the strengths of all the mentioned indicators [2]. Figure 1 shows the incidence of breast cancer cases in different countries like South America, North America, East Asia and South-Central Asia. Cancer is a broad term that refers to a group of diseases that share a common characteristic: the abnormal proliferation of body cells or cell multiplication so the body cannot regulate it properly. Unpacking that way, cancerous cells depart from the regulated growth and division orders that healthy cells follow and divide and multiply, basically, at an exorbitant rate, which ends up in the formation and enlargement of malignant tumors or neoplasms [14]. There are various types of cancers

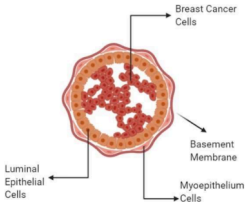


Fig. 1. Cancer Cell

that are named on the basis of the location of concurrence in the human body. For example, lung cancer, breast cancer, colorectal cancer, prostate cancer and skin cancer are the most common type of cancers worldwide. Figure 2 shows some common types of cancers and how common they are based on the number of cases worldwide per year. Cancer was a disease that led to the genome material which was started due to cell division and growth pathway interference. Alterations in genes were the cause of these mutations. These genetic modifications identify the cells for the new job and the specific task, they emphasize controlled cellular growth as well as its checks cells that are healthy contain a complex system of checks and balances which is supposed to govern orderly growth and division; however, in cancer cells, this is disturbed by genetic mutations and leads to uncontrolled growth. In the process, the cells as a whole turn into mutineers, breaching the jurisdiction of usual laws and proliferating endlessly, and eventually become tumors or spread to other

B. AI SIMILARITY REPORT

Breast_Cancer_SL.docx

ORIGINALITY REPORT

9%	3%	7%	3%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Tulane University Student Paper	1%
2	Sumit Garethiya, Himanshu Agrawal, Shilpa Gite, Suresh. V, Amit Kudale, Gaurav Wable, Girishchandra R. Yendargaye. "Affordable system for alerting, monitoring and controlling heat stroke inside vehicles", 2015 International Conference on Industrial Instrumentation and Control (ICIC), 2015 Publication	1%
3	Abdul, Abdullah. "Using Machine Learning Algorithms to Find Novel Biomarkers for Breast Cancer Using RNA-Seq Dataset", Morgan State University, 2023 Publication	1%
4	H L Gururaj, Francesco Flammini, V Ravi Kumar, N S Prema. "Recent Trends in Healthcare Innovation", CRC Press, 2025 Publication	1%
5	Mahmoud Darwich, Magdy Bayoumi. "An evaluation of the effectiveness of machine learning prediction models in assessing breast cancer risk", Informatics in Medicine Unlocked, 2024 Publication	1%
6	doctorpenguin.com Internet Source	1%
7	scienceofbiogenetics.com Internet Source	1%
	Submitted to Nottingham Trent University	

A Machine Learning-Based Predictive Model for Early Detection of Breast Cancer

1st Aaryan Tamhane
Computer Science
Symbiosis Institute of Technology
Nagpur India
aaryantamhane29@gmail.com

2nd Advait Kulkarni
Computer Science
Symbiosis Institute of Technology
Nagpur India
kulkarniadvait93@gmail.com

3rd Soham Shrawankar
Computer Science
Symbiosis Institute of Technology
Nagpur India
sohamshrawankar@gmail.com

4th Sagarkumar Badhiye
Computer Science
Symbiosis Institute of Technology
Nagpur India
sagarkumar.badhiyc@sitnagpur.siu.edu.in

5th Pradnya Borkar
Computer Science
Symbiosis Institute of Technology
Nagpur India
pradnyaborkar2@gmail.com

***Abstract*—Breast cancer is one of the biggest health risks today, and early diagnosis is essential for a better prognosis of the disease. Data-driven approaches such as machine learning emerge successful in diagnosis and breast cancer prediction. This section assesses several general classifiers for prediction such as Random forest (RF), Logistic Regressions (LR) etc. algorithms explored on breast cancer datasets. The dataset is a fine needle aspiration (Biopsy) dataset namely Wisconsin Breast Cancer dataset, which was obtained from the UCI Machine Learning (ML) database. Discussions on the comparisons of different algorithms, feature selection methods, and performance measures are also covered.**

***Index Terms*—Breast-cancer, Machine-learning, Classification algorithms, SVM, Random forest, Feature selection, Data mining.**

I. Introduction

Breast-cancer is a serious and common type of cancer that is a great cause of death among women worldwide [1]. World Health Organization specifies it as a leading cause of death, responsible for around the fifteenth percent of all cancer death rates in women [5]. Timely and accurate detection followed by judicious handling of the disease path in the first stages, improves prognosis and survival rates- vastly [1]. While the accuracy, consistency, and efficiency of traditional diagnostic methods, such as mammography, are problematic in the early stages of the disease, non-invasive methods powered by modern technologies usually demonstrate the strengths of all the mentioned indicators [2]. Figure 1 shows the incidence of breast cancer cases in different countries like South America, North America, East Asia and South-Central Asia. Cancer is a broad term that refers to a group of diseases that share a common characteristic: the abnormal proliferation of body cells or cell multiplication so the body cannot regulate it properly. Unpacking that way, cancerous cells depart from the regulated growth and division orders that healthy cells follow and divide and multiply, basically, at an exorbitant rate, which ends up in the formation and enlargement of malignant tumors or neoplasms [14]. There are various types of cancers

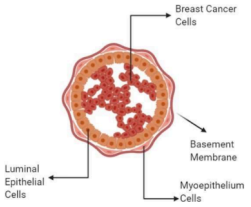
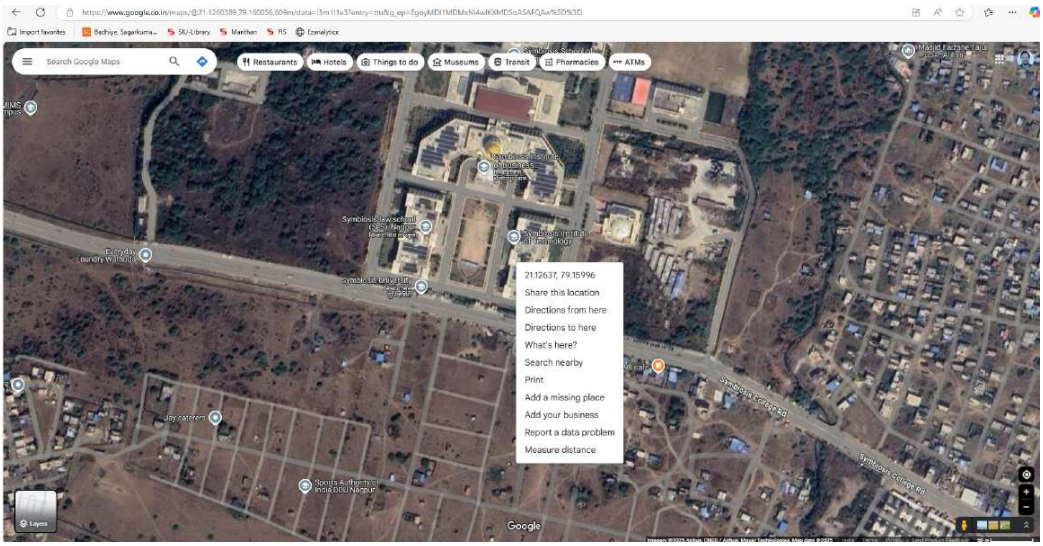
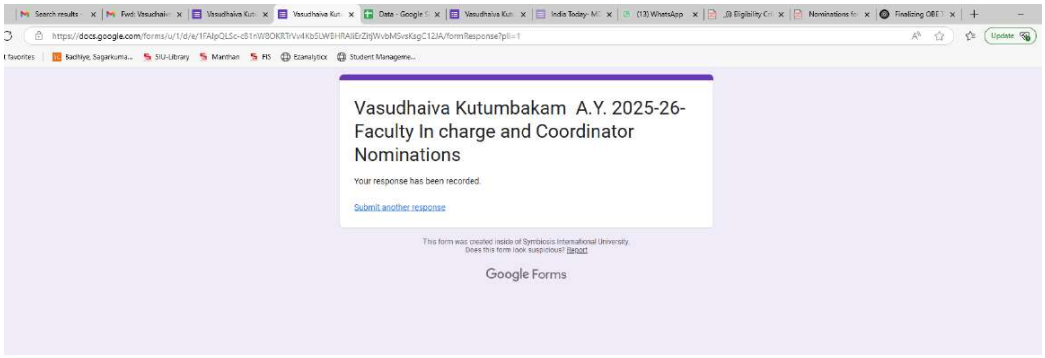


Fig. 1. Cancer Cell

that are named on the basis of the location of concurrence in the human body. For example, lung cancer, breast cancer, colorectal cancer, prostate cancer and skin cancer are the most common type of cancers worldwide. Figure 2 shows some common types of cancers and how common they are based on the number of cases worldwide per year. Cancer was a disease that led to the genome material which was started due to cell division and growth pathway interference. Alterations in genes were the cause of these mutations. These genetic modifications identify the cells for the new job and the specific task, they emphasize controlled cellular growth as well as its checks cells that are healthy contain a complex system of checks and balances which is supposed to govern orderly growth and division; however, in cancer cells, this is disturbed by genetic mutations and leads to uncontrolled growth. In the process, the cells as a whole turn into mutineers, breaching the jurisdiction of usual laws and proliferating endlessly, and eventually become tumors or spread to other

C. PROJECT SCREENSHOTS




Vasudhaiva Kutumbakam A.Y. 2025-26- Faculty In charge and Coordinator Nominations
Your response has been recorded.
Submit another response
This form was created inside of Symbiosis International University.
Does this form look suspicious? Report
Google Forms